Saga Patterns - Choreography, Orchestration and Simulation

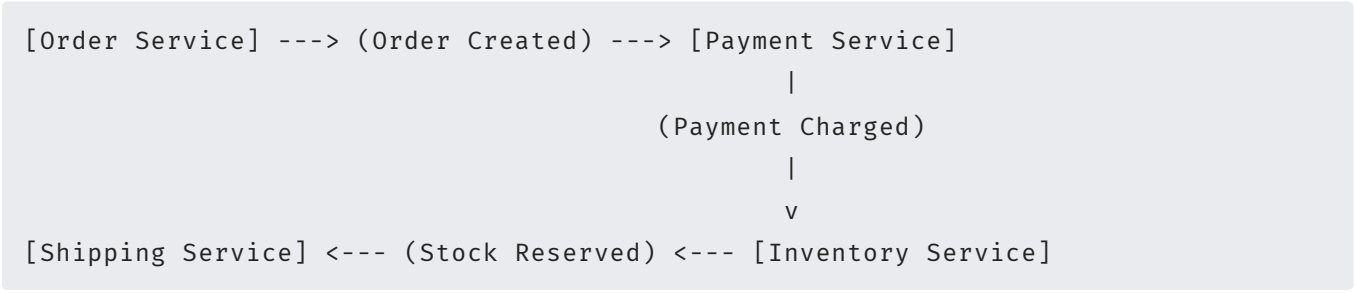
In distributed microservice architectures, managing data consistency across multiple database instances without relying on blocking Two-Phase Commit (2PC) transactions is a core challenge. The **Saga Pattern** solves this problem by breaking a global transaction into a sequence of local transactions. When executing a Saga, you have two distinct coordination styles: **Choreography** and **Orchestration**.

1. The Core Concept: What is a Saga?

A **Saga** is a sequence of localized microservice transactions. Every local transaction updates data within its own single database boundary and emits an event or message to trigger the next step. Because distributed systems lack immediate global consistency, Sagas rely on **eventual consistency**. If step 3 of a 5-step transaction fails, the system cannot perform a traditional database rollback. Instead, it must sequentially execute explicit **Compensating Transactions** in reverse order (C_2, C_1) to undo the already committed state mutations (T_1, T_2).

2. Choreography-Based Saga (Decentralized & Event-Driven)

In a **Choreography** flow, there is no central orchestrator or manager. Services communicate directly by subscribing to and emitting domain events via a message broker (e.g., Kafka, RabbitMQ).



- **Happy Path:**
  1. `Order Service` creates an order in `PENDING` state and emits `OrderCreated`.
  2. `Payment Service` consumes `OrderCreated`, charges the credit card, and emits `PaymentCharged`.
  3. `Inventory Service` consumes `PaymentCharged`, reserves stock, and emits `StockReserved`.
  4. `Shipping Service` consumes `StockReserved` and generates a label.
- **Failure Handling (Compensations):**
  If `Inventory Service` is out of stock, it emits `StockReservationFailed`. The `Payment Service` consumes this event and issues a refund (`Compensating Transaction`).

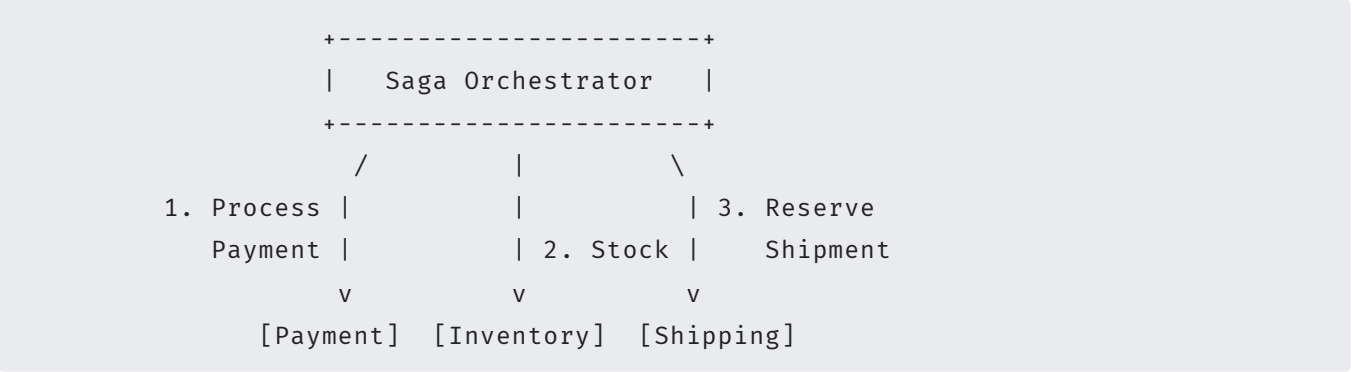
Trade-offs:

- **Pros:** Highly decoupled, low latency, no single point of failure or centralized logic bottleneck.

- **Cons:** Hard to visualize the end-to-end workflow; risk of circular dependencies; high cognitive load to track state across 10+ services.

3. Orchestration-Based Saga (Centralized Coordinator)

In an **Orchestration** flow, a dedicated centralized service (the **Orchestrator** or Saga Execution Coordinator) dictates the control flow by explicitly commanding participating services what actions to execute via direct async API calls or targeted message queues.



- **Happy Path:**
  1. `Order Service` sends `CreateOrderCommand` to `OrderSagaOrchestrator`.
  2. Orchestrator calls `Payment Service`. Upon receiving `Success`, it calls `Inventory Service`.
  3. Orchestrator receives `Success` from `Inventory` and calls `Shipping Service`.
- **Failure Handling (Compensations):**
  If `Inventory Service` fails, the Orchestrator catches the error, looks up its state machine definition, and directly invokes `RefundPayment` on `Payment Service`.

Trade-offs:

- **Pros:** Single source of truth for workflow state; easy to monitor and audit; straightforward state machines; ideal for complex multi-step workflows.
- **Cons:** Risk of centralization (business logic leaking into the orchestrator); additional infrastructure requirement (e.g., Temporal, AWS Step Functions, Zeebe).

4. Choreography vs. Orchestration Comparison

Feature	Choreography	Orchestration
Control Flow	Decentralized (Event-driven)	Centralized (Command-driven)
Coupling	Loose (Services only know local events)	Medium (Orchestrator knows service interfaces)
State Tracking	Distributed across service databases	Unified inside the Orchestrator state engine
Complexity Growth	Exponential with scale ($> 5-7$ services gets chaotic)	Linear (Easier to maintain complex workflows)
Ideal Use Case	Simple 2–4 step workflows, greenfield microservices	Complex Enterprise workflows, long-running transactions

Interactive Saga State Machine Simulator

The following simulation demonstrates the step-by-step state transitions of an **Orchestration Saga**, including forward execution and automatic rollback via compensating transactions when a mid-sequence step fails.